

Group Project Assignment 2: Data Modeling

1. The scope of our societal problem: We want to build a database that shows us the implementation of AI in entry level employment opportunities specifically for new graduates. The database should be able to answer questions about changes in entry level job demand, which jobs are heavily affected by AI, and how employers respond to AI implementation.

2. Data interaction:

- a. A graduate can apply for many job postings, and one job posting can receive applications from many graduates
- b. An employer can post many job postings, but each job posting belongs to one employer
- c. Each job posting is for one type of entry level job, but the same type of job can be posted many times by different employers
- d. An employer can adopt many AI technologies, and one AI technology can be adopted by many employers. Each employer adopts it on its own date and uses it in its own way
- e. AI technology can affect many types of entry level jobs, and one type of entry level job can be affected by many AI technologies
- f. An employer can offer many workplace training programs, but each program belongs to one employer. A training program can focus on one AI technology
- g. When an employer adopts AI, this can change the number of entry level jobs it offers

3. The entities:

- a. Graduate
- b. Employer
- c. Entry Level Job
- d. Job Posting
- e. AI Technology
- f. Workplace Training

4. Connections between entities:

- a. **Employer → posts → Job Posting:** one employer can post many job postings, but each job posting belongs to one employer (one to many).
- b. **Entry Level Job → is advertised in → Job Posting:** one type of entry level job can appear in many job postings, but each job posting is for one type of job (one to many).
- c. **Employer → provides → Workplace Training:** one employer can provide many training programs, but each program belongs to one employer (one to many).
- d. **AI Technology → is taught in → Workplace Training:** one AI technology can be the focus of many training programs, and a training program focuses on zero or one AI technology (one to many, optional).
- e. **Employer → adopts → AI Technology:** one employer can adopt many AI technologies, and one AI technology can be adopted by many employers (many to many).
- f. **Graduate → applies for → Job Posting:** one graduate can apply for many job postings, and one job posting can receive applications from many graduates (many to many).
- g. **AI Technology → affects → Entry Level Job:** one AI technology can affect many entry level jobs, and one entry level job can be affected by many AI technologies (many to many).

5. Attributes per entity:

a. Graduate: We're tracking GraduateID, FirstName, LastName, Email, GraduationYear, DegreeField, and University. From these columns we get all the relevant information we need about the graduate.

b. Employer: We keep track of three attributes for this entity: EmployerID, CompanyName and CompanySize. This way we are able to see what companies are posting jobs, and compare small, medium and large companies.

c. Job Posting: There are quite some variables to keep track of for this entity. JobPostingID, EmployerID (FK), EntryLevelJobID (FK), JobTitle, JobDescription, PostedDate, ClosingDate, MinSalary, MaxSalary, and Location. We have every detail about the job

posting listed: who is posting it, where it's being posted, how much it pays, and which type of entry level job it is.

d. Entry Level Job: EntryLevelJobID and JobName. This lets us track demand trends over time for each type of job, by counting the job postings for each type. We don't store an average salary here, because it can be calculated from MinSalary and MaxSalary in Job Posting, and storing it twice could cause the two to go out of sync.

e. AI Technology: The AI technology itself is captured by AITechnologyID and TechnologyName. The details of how each employer implements AI (AdoptionDate, ImplementationType) and how strongly it affects each job (ImpactLevel) are stored in the bridge tables, because these are different for every employer and every job.

f. Workplace Training: TrainingID, EmployerID, AITechnologyID, TrainingName, TrainingType, Duration, and Format. This would enable us to observe what kind of training employers give to their employees, and whether the training is about an AI technology.

6. Define your PKs and FKs:

a. Our entities each have their own corresponding Primary Key: GraduateID, EmployerID, JobPostingID, EntryLevelJobID, AITechnologyID, and TrainingID. These ensure that each record is easily identifiable.

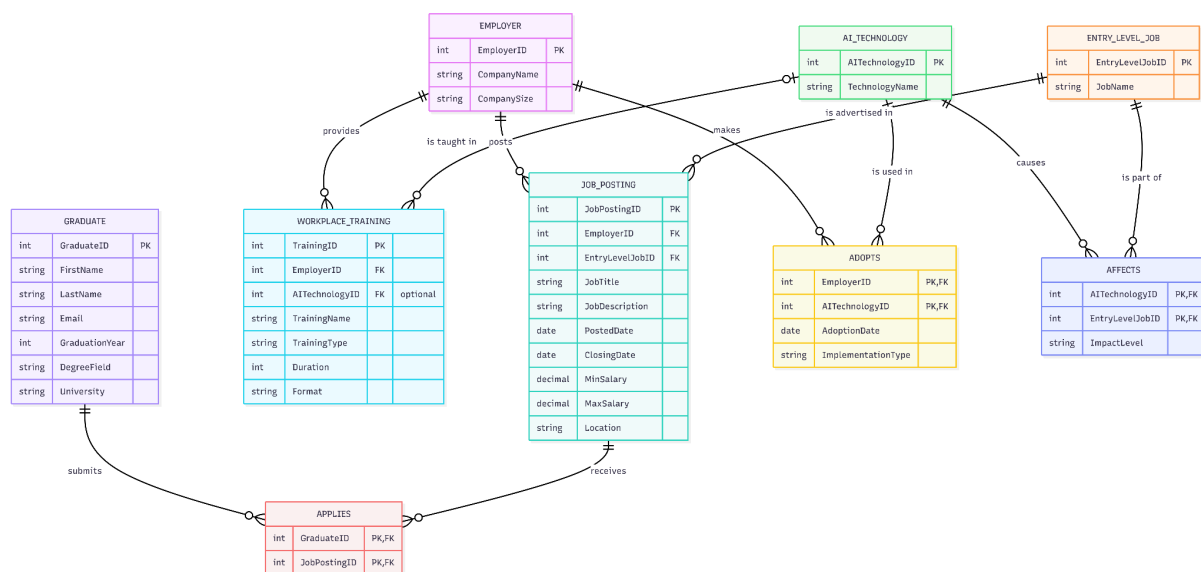
b. The child tables will be connected back to the parent tables using Foreign Keys. We assign the FK to the "many" table, because each row there belongs to exactly one row in the "one" table:

- JobPosting.EmployerID references Employer.EmployerID
- JobPosting.EntryLevelJobID references EntryLevelJob.EntryLevelJobID
- WorkplaceTraining.EmployerID references Employer.EmployerID
- WorkplaceTraining.AITechnologyID references AITechnology.AITechnologyID (can be empty if the training is not about AI)

c. There were also a few Many-to-Many relationships, so we had to create bridge (junction) tables to decompose those relationships. Every bridge table has a PK that is composed of the two FKs that it bridges:

- Applies (GraduateID, JobPostingID): connects Graduates to the Job Postings they apply for.
- Adopts (EmployerID, AITechnologyID): links Employers to the AI technology that they have adopted. This table also stores AdoptionDate and ImplementationType, because every employer adopts AI on its own date and in its own way.
- Affects (AITechnologyID, EntryLevelJobID): relates AI Technologies to Entry Level Jobs that they affect. This table also stores ImpactLevel, because AI affects every job differently.

7. ERD



- tool used: mermaid.live

8. Database normalization

We normalized our database so every piece of information is stored only once. Otherwise, one change means updating many rows, and missing one makes the data contradict itself. Below, we start from one big table and fix it step by step, using made-up data.

Starting table: Raw_Applications

GraduateID	GraduateName	JobPostingID	JobTitle	EmployerID	CompanyName	CompanySize	SalaryRange	AITechnologies
G1	Emma Jansen	P10	Junior Data Analyst	E1	DataCorp	Large	€2,800-€3,200	ChatGPT, Copilot
G2	Yusuf Kaya	P10	Junior Data Analyst	E1	DataCorp	Large	€2,800-€3,200	ChatGPT, Copilot
G1	Emma Jansen	P11	Trainee Accountant	E2	GreenBank	Medium	€2,500-€2,900	UiPath
G2	Yusuf Kaya	P12	Marketing Assistant	E1	DataCorp	Large	€2,600-€3,000	ChatGPT, Copilot

Step 1: First Normal Form (1NF), atomic values

1NF means one value per cell. Three columns break this: GraduateName (first and last name together), SalaryRange (two numbers stored as text, so we can't calculate with it) and AITechnologies (a list). We split the first two into separate columns and moved the AI list into its own table, with one row per tool. Since the tools belong to the employer and not the application, that table uses EmployerID.

After 1NF: Applications (PK: GraduateID + JobPostingID)

GraduateID	FirstName	LastName	JobPostingID	JobTitle	EmployerID	CompanyName	CompanySize	MinSalary	MaxSalary
G1	Emma	Jansen	P10	Junior Data Analyst	E1	DataCorp	Large	2800	3200
G2	Yusuf	Kaya	P10	Junior Data Analyst	E1	DataCorp	Large	2800	3200
G1	Emma	Jansen	P11	Trainee Accountant	E2	GreenBank	Medium	2500	2900
G2	Yusuf	Kaya	P12	Marketing Assistant	E1	DataCorp	Large	2600	3000

After 1NF: Employer_AI (PK: EmployerID + AITechnology)

EmployerID	AITechnology
E1	ChatGPT
E1	Copilot
E2	UiPath

Step 2: Second Normal Form (2NF), no partial dependencies

2NF means every column has to depend on the whole PK, which here is GraduateID + JobPostingID. But the names only depend on GraduateID, and the job and company details

only depend on JobPostingID. That's why "Emma Jansen" and posting P10 show up twice. We split it into three tables, which leaves Applies with just the two keys.

Graduate (PK: GraduateID)

GraduateID	FirstName	LastName
G1	Emma	Jansen
G2	Yusuf	Kaya

Job_Posting (PK: JobPostingID)

JobPostingID	JobTitle	EmployerID	CompanyName	CompanySize	MinSalary	MaxSalary
P10	Junior Data Analyst	E1	DataCorp	Large	2800	3200
P11	Trainee Accountant	E2	GreenBank	Medium	2500	2900
P12	Marketing Assistant	E1	DataCorp	Large	2600	3000

Applies (PK: GraduateID + JobPostingID)

GraduateID	JobPostingID
G1	P10
G2	P10
G1	P11
G2	P12

Our own first draft had the same problem. The AI Implementation table mixed the technology with adoption and impact details, so it needed a row for every combination of technology, employer and job:

AITechnologyID	TechnologyName	EmployerID	AdoptionDate	ImplementationType	EntryLevelJobID	ImpactLevel
T1	ChatGPT	E1	2025-03-01	Customer service chatbot	J1	High
T1	ChatGPT	E1	2025-03-01	Customer service chatbot	J2	Medium
T1	ChatGPT	E2	2026-01-15	Report writing	J1	High

TechnologyName only depends on AITechnologyID, AdoptionDate and ImplementationType only on AITechnologyID + EmployerID, and ImpactLevel only on AITechnologyID + EntryLevelJobID. So we split it into three tables:

AI_Technology (PK: AITechnologyID)

AITechnologyID	TechnologyName
T1	ChatGPT

Adopts (PK: EmployerID + AITechnologyID)

EmployerID	AITechnologyID	AdoptionDate	ImplementationType
E1	T1	2025-03-01	Customer service chatbot
E2	T1	2026-01-15	Report writing

Affects (PK: AITechnologyID + EntryLevelJobID)

AITechnologyID	EntryLevelJobID	ImpactLevel
T1	J1	High
T1	J2	Medium

The Employer_AI table from Step 1 becomes Adopts once the tool names are replaced by AITechnologyID.

Step 3: Third Normal Form (3NF), no transitive dependencies

3NF means non-key columns can't depend on other non-key columns. In Job_Posting, CompanyName and CompanySize depend on EmployerID instead of the key (JobPostingID → EmployerID → CompanyName). So DataCorp's details are stored twice, and a change would mean updating all of its postings.

Before 3NF: Job_Posting

JobPostingID	JobTitle	EmployerID	CompanyName	CompanySize	MinSalary	MaxSalary
P10	Junior Data Analyst	E1	DataCorp	Large	2800	3200
P11	Trainee Accountant	E2	GreenBank	Medium	2500	2900
P12	Marketing Assistant	E1	DataCorp	Large	2600	3000

We moved the employer details into their own Employer table and kept only EmployerID as a FK in Job_Posting:

After 3NF: Employer (PK: EmployerID)

EmployerID	CompanyName	CompanySize
E1	DataCorp	Large
E2	GreenBank	Medium

After 3NF: Job_Posting (PK: JobPostingID)

JobPostingID	JobTitle	EmployerID	MinSalary	MaxSalary
P10	Junior Data Analyst	E1	2800	3200
P11	Trainee Accountant	E2	2500	2900
P12	Marketing Assistant	E1	2600	3000

The same goes for JobName: it depends on the job type, not the posting, so it's stored in Entry Level Job. After these steps, every table matches our ERD and each piece of information is stored only once.

9. Scope check

Our final database can answer all three scope questions:

- **Demand over time:** count job postings per month (PostedDate), per job type (EntryLevelJobID) and per company size (CompanySize).
- **Jobs most affected by AI:** combine Affects and Entry Level Job and sort by ImpactLevel.
- **Employer response to AI:** compare the number of postings before and after the AdoptionDate in Adopts, and use Workplace Training to see if employers train staff on the AI they adopted.

After adding CompanySize, EntryLevelJobID and AITechnologyID, and moving the AI details into the bridge tables, we checked normalization again. Every column holds one value and depends only on its own table's key, so the database is still in 3NF.